SupportVectors · Follow publication

This member-only story is on us. Upgrade to access all of Medium.

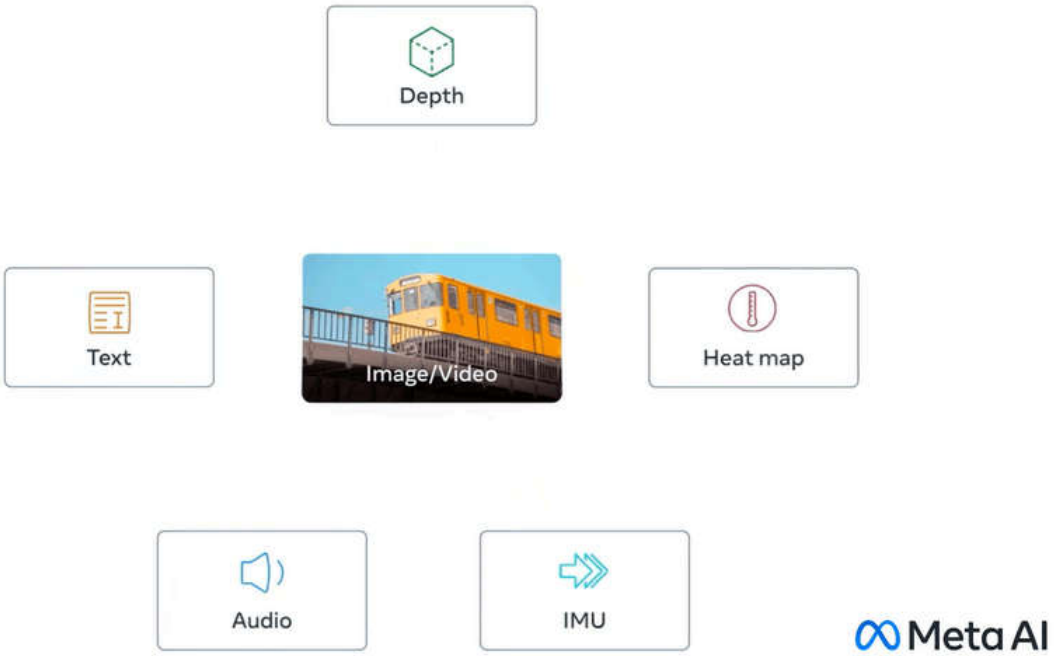
Member-only story

Embedding Multi-Modal Data with ImageBind

6 min read · Dec 19, 2023

Athira B Follow

Listen Share ••• More



Source: Meta

A while ago, I was trying to experiment with multimodal data for similarity search. This was my approach - I converted all my data, whether it was

images, audio, or video, into text. The goal was to establish a common ground, enabling the generation of embeddings using a Sentence Transformer.

Early this year, Meta released ImageBind - a model that redefines the boundaries of multimodal learning. **This approach revolves around creating a unified embedding across six diverse modalities: images, text, audio, depth, thermal, and IMU data**. The model harnesses the capabilities of state-of-the-art vision-language models on a large scale.

The core idea is that an image can invoke many sensory experiences, connecting visual information with auditory and even tactile stimuli. Many existing methods focus on learning from images paired with just one type of data, like text or audio. However, these methods are limited because they only work with the pairs they were trained on. For example, if a method learns from video and audio, it can't be used directly for tasks involving images and text.

What sets ImageBind apart is its ability to forge a joint embedding across all modalities — image being the binding factor. ImageBind uses large-scale image-text pairs from the web and pairs them with naturally occurring data, like video-audio or image-depth combinations. The embeddings of each modality are aligned with the embedding of images. The resulting embedding space exhibits robust zero-shot behavior- which means that the model can understand and connect different types of information without being explicitly trained on every possible combination.

Let us dive into a practical application that takes the embeddings to the next level - using **cosine similarity** for search functionality. Since our dataset is very small, cosine similarity would be a good way to test things out. For larger datasets, we can employ more powerful mechanisms like FAISS.

Setting Up ImageBind

ImageBind is not available as a Python library yet, so we need to clone the official Github repo and work along with the code provided there.

Note! *While generating embeddings using ImageBind is straightforward, setting it up can be quite cumbersome if you run into version conflicts between packages based on your Python version. Feel free to update the requirements.txt file to address these conflicts.*

```
# Create a venv and clone the official repo
conda create --name imagebind python=3.8 -y
conda activate imagebind
git clone https://github.com/facebookresearch/ImageBind
cd ImageBind

# Install all dependencies using below command
pip install .

# For windows users, you might need to install soundfile for reading/writing au
pip install soundfile

# If you encounter error "Could not build wheels for mayavi, which is required
python -m pip install vtk==9.2.4 git+https://github.com/enthought/mayavi.git@ma

# If you encounter "Unable to install Cartopy: Failed building wheel for Cartop
conda install -c conda-forge cartopy
```

Data

To represent **text** data, we input the following: [“A dog”, “A car”, “A bird”]. For the remaining modalities, we will use the sample data provided in *.assets/*. There are three **audio** clips: the chirping of a bird, a barking dog, and the honking of a car. We also have these **images**: a dog, a bird, and a car.

Code to Generate Embeddings

We will use the same code sample provided in the official ***Github***.

```
# Importing the necessary libraries and code from ImageBind repo
from imagebind import data
import torch
from imagebind.models import imagebind_model
from imagebind.models.imagebind_model import ModalityType
from PIL import Image
from IPython.display import display

# These assets will be present in the Github repo
text_list=["A dog", "A car", "A bird"]
image_paths=[".assets/dog_image.jpg", ".assets/car_image.jpg", ".assets/bird_im
audio_paths=[".assets/dog_audio.wav", ".assets/car_audio.wav", ".assets/bird_au

device = "cuda:0" if torch.cuda.is_available() else "cpu"

# Instantiate model
```

```
model = imagebind_model.imagebind_huge(pretrained=True)
model.eval()
model.to(device)

# Load data of each modality and transform them to generate embeddings
inputs = {
    ModalityType.TEXT: data.load_and_transform_text(text_list, device),
    ModalityType.VISION: data.load_and_transform_vision_data(image_paths, devic
    ModalityType.AUDIO: data.load_and_transform_audio_data(audio_paths, device)
}

# Generate embeddings of each modality
with torch.no_grad():
    embeddings = model(inputs)

print(embeddings)
```

```
{'text': tensor([[ 0.2054, -5.2253, -2.9712,  ...,  3.0710, -4.6814, -0.9082],
         [-0.4342, -0.9050, -4.2879,  ...,  3.6109, -3.5757, -1.1599],
         [-1.3476, -1.5732, -0.7386,  ...,  3.0858, -2.6202,  0.5023]],
        device='cuda:0'),
 'vision': tensor([[ 0.0020, -0.0281,  0.0052,  ...,  0.0134,  0.0024,  0.0146],
         [ 0.0259,  0.0054,  0.0399,  ..., -0.0253, -0.0048, -0.0196],
         [ 0.0186,  0.0058,  0.0546,  ...,  0.0043,  0.0096, -0.0092]],
        device='cuda:0'),
 'audio': tensor([[-0.0282, -0.4923,  1.0057,  ...,  0.3342, -0.1340, -0.6134],
         [ 0.7091,  0.2072, -1.0133,  ...,  0.4945, -0.2343,  0.4717],
         [ 0.2213, -0.2542,  0.3885,  ..., -0.2424,  0.0875,  0.1380]],
        device='cuda:0')}
```

Embeddings Generated by ImageBind

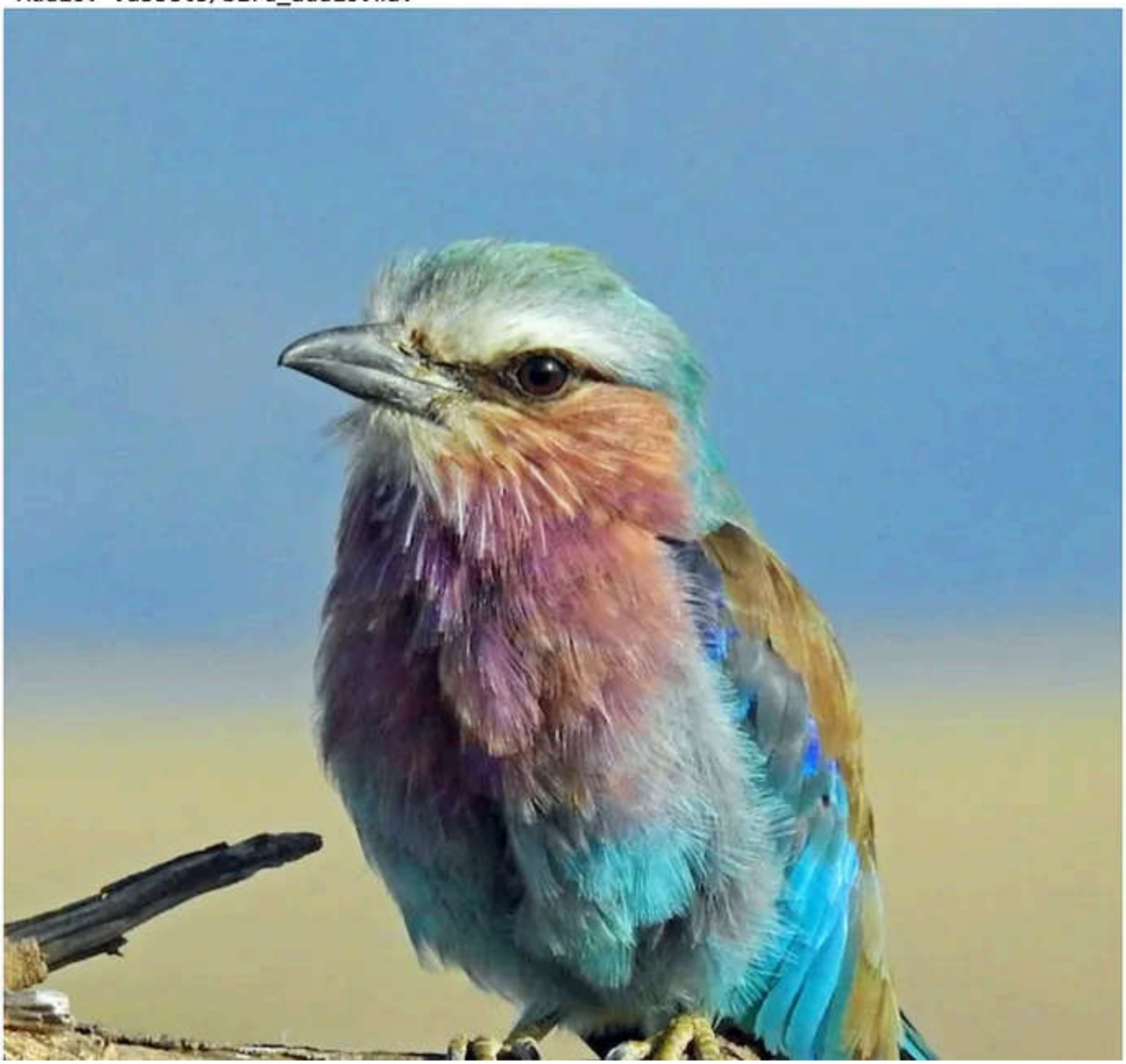
Using cosine similarity to test search functionality

Now let us use cosine similarity to find the top 3 similar results based on an input.

```
# User Input
user_input = ["I love parrots"]

# Embedding the input query into the Text modality embedding space
user_input_load = {
    ModalityType.TEXT: data.load_and_transform_text(user_query, device)}
with torch.no_grad():
    user_input_embeddings = model(user_input_load)
    user_input_embeddings = user_input_embeddings[ModalityType.TEXT]

# Calculate cosine similarities between user query and each modality's embeddir
text_similarities = torch.nn.functional.cosine_similarity(user_input_embeddings
```

```
image_similarities = torch.nn.functional.cosine_similarity(user_input_embedding
audio_similarities = torch.nn.functional.cosine_similarity(user_input_embedding

# Find top 3 results for each modality based on highest cosine similarity
top3_text_indices = torch.topk(text_similarities, 3).indices
top3_image_indices = torch.topk(image_similarities, 3).indices
top3_audio_indices = torch.topk(audio_similarities, 3).indices

# Retrieve top matching text, image, and audio based on their indices
top_text_results = [text_list[i] for i in top3_text_indices]
top_image_results = [image_paths[i] for i in top3_image_indices]
top_audio_results = [audio_paths[i] for i in top3_audio_indices]

# Print the top search results for each modality
print("Top Matches\n")
print(f"Text: {top_text_results[0]}")
print(f"Image: {top_image_results[0]}")
print(f"Audio: {top_audio_results[0]}")

# Display the top image result
img = Image.open(top_image_results[0])
display(img)
```

```
Top Matches

Text: A bird
Image: .assets/bird_image.jpg
Audio: .assets/bird_audio.wav
```



Results for “I love parrots”

This looks great! The results show the top result from each modality and it perfectly understands the input we are giving. Now, let’s try another query.

```
# User Input
user_input = ["I have a Ferrari"]

# Rest of code remains the same
```

```
Top Matches

Text: A car
Image: .assets/car_image.jpg
Audio: .assets/car_audio.wav
```

Results of input “I have a Ferrari”

The results are spot on again! Now let’s try something that is not present in the data.

```
# User Input
user_input = ["Kittens are so cute"]
```

```
Top Matches

Text: A car
Image: .assets/bird_image.jpg
Audio: .assets/car_audio.wav
```

Results for input “Kittens are so cute”

It is strange that “A car” and car audio appear as top results. We should also note that there is no mention of dogs, who are potentially the top candidates. But I guess birds are cute too.

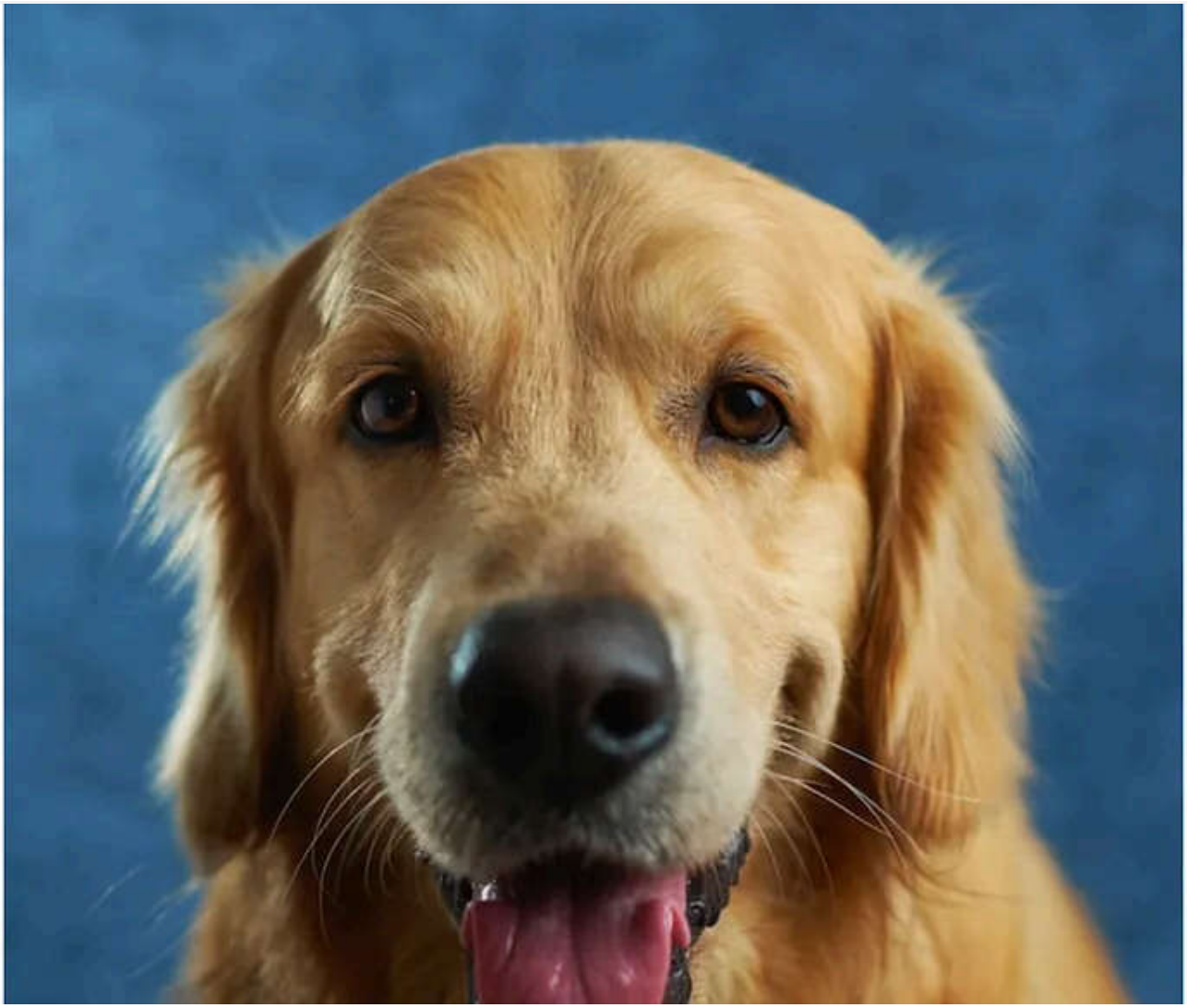
Let’s give it another try.

```
# User Input
user_input = ["She trotted to the park"]
```

```
Top Matches

Text: A car
```

Open in app ↗

Medium



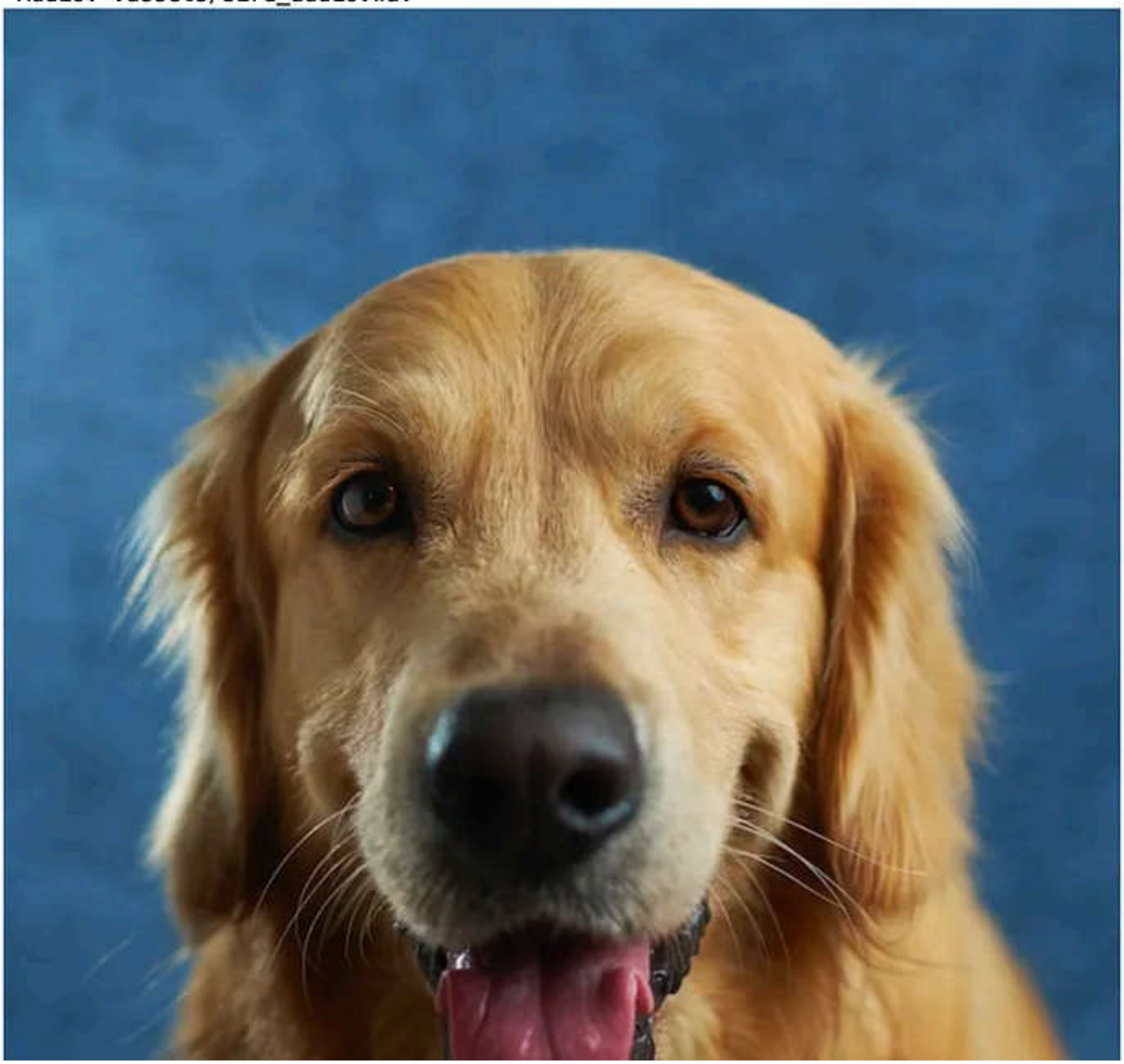
Results for input “She trotted to the park”

“A car” has appeared as the top result again. The other two responses look correct. Does the model think that cars trot, or is it the “to the park”? Maybe.

I’m going to give it one last try.

```
user_input = ["She mimicked his mannerisms"]
```

```
Top Matches

Text: A car
Image: .assets/dog_image.jpg
Audio: .assets/bird_audio.wav
```



Results for input “She mimicked his mannerisms”

Once again, “A car” has come up as the top result, even though the context is completely unrelated. The second and third results look fine. My best guess is that when the model is unsure about the input, it responds inaccurately even when better results exist.

Observation

ImageBind is showing promising results as is evident from the above examples. I also plan to give the model a large dataset and use FAISS the next time to see how the results look. Even though the authors of the paper claim state-of-the-art performance on several benchmarks, our negative test cases underscore the need for a closer examination of the model’s capabilities. However, it is truly remarkable

that we now have a model that can represent multimodal information in a single embedding space.

To learn more, visit Meta or read the research paper.

Artificial Intelligence | Machine Learning | Multi Modal Learning | Vector Embeddings | Similarity Search

Follow

Published in SupportVectors

7 followers · Last published Dec 19, 2023

Blogs and articles from SupportVectors.ai

Follow

Written by Athira B

20 followers · 9 following

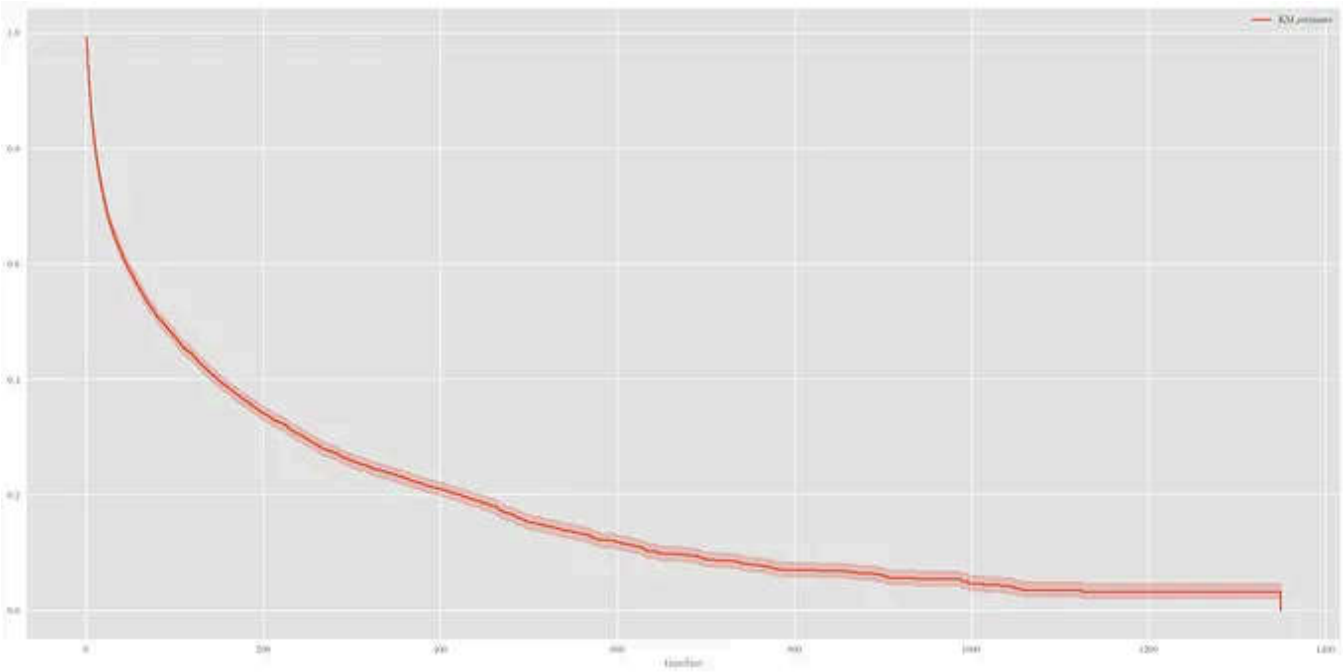
Data Scientist

No responses yet

H Hanine Benamor

What are your thoughts?

More from Athira B and SupportVectors



In SupportVectors by Chandar Lakshminarayan

Survival modeling—a case study

This short article explains a survival modeling problem we implemented recently for one of our clients. It was interesting and instructive...

Oct 14, 2023 3

In SupportVectors by Chandar Lakshminarayan

Gradient descent of the mind

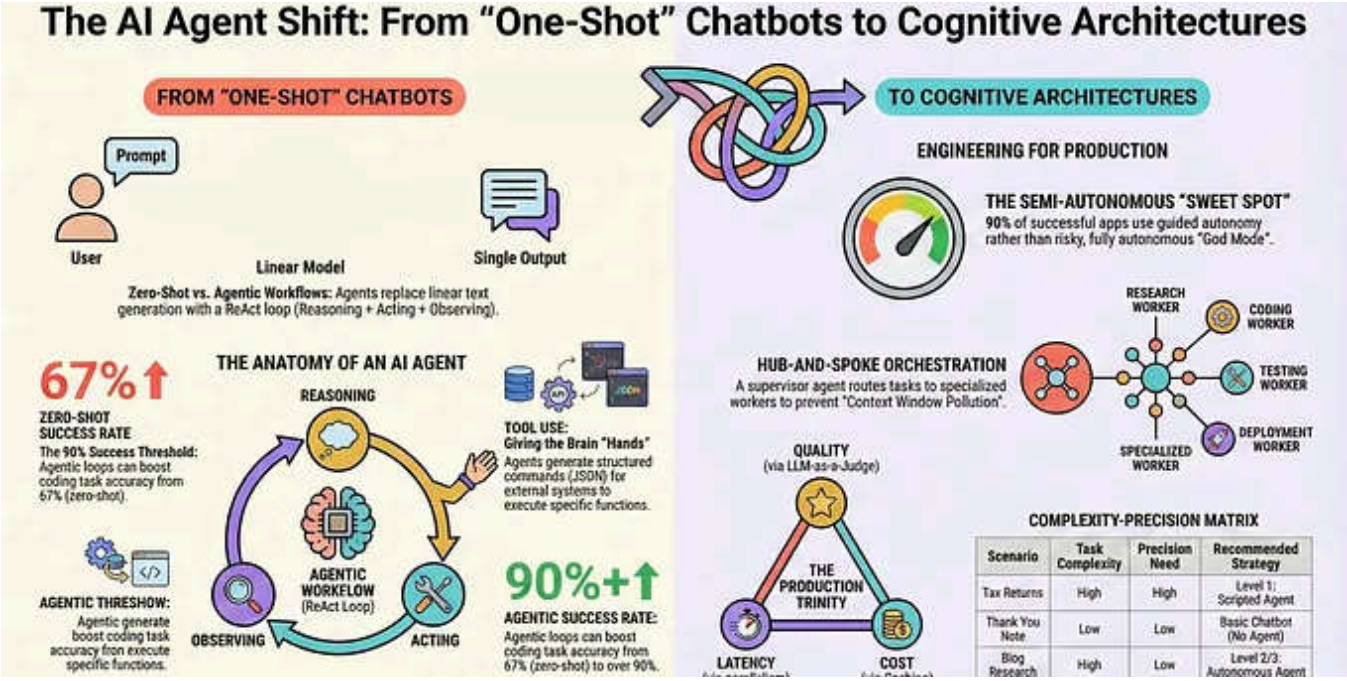
Introduction

Aug 23, 2023 10 1

See all from Athira B

See all from SupportVectors

Recommended from Medium



In Level Up Coding by Gaurav Shrivastav

Building AI Agents in 2026: Chatbots to Agentic Architectures

This is the engineering blueprint for building production-ready agentic systems that actually work.

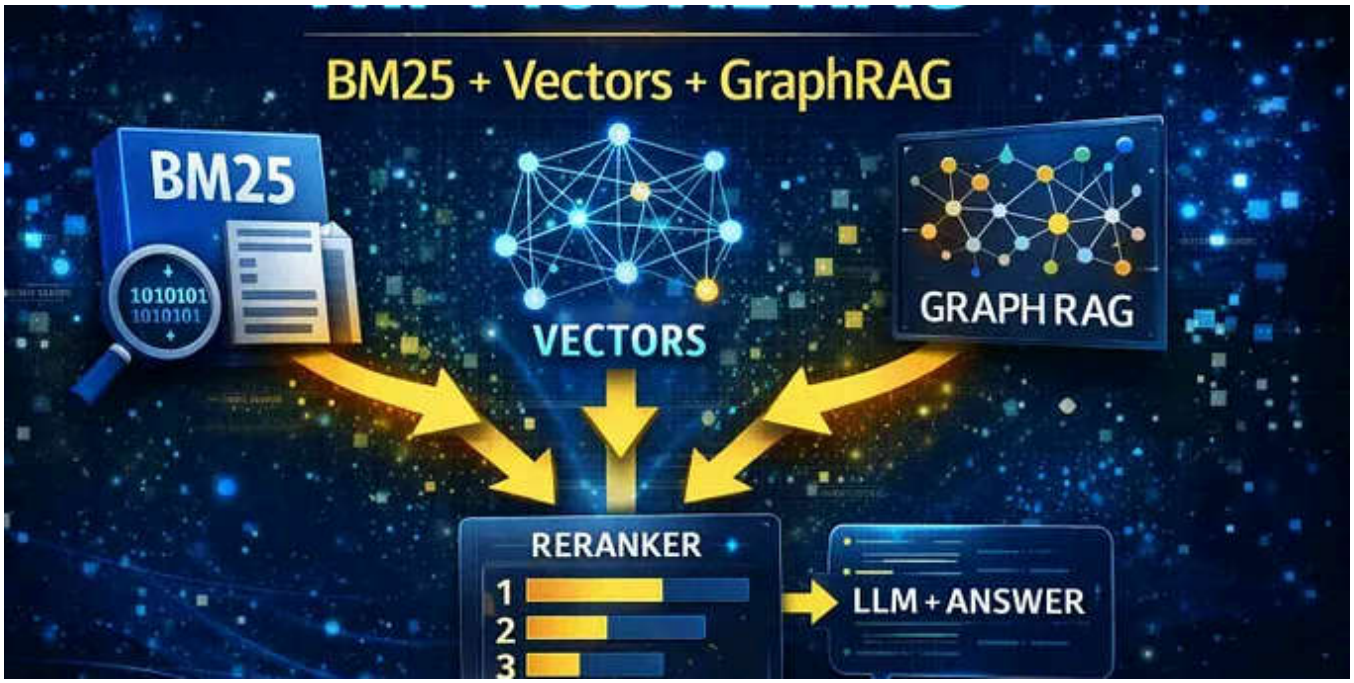
Jan 22 314 1

Joe Njenga

I Tried New Claude Code Ollama Workflow (It's Wild & Free)

Claude Code now works with Ollama, which takes the game to the next level for developers who want to work locally or need flexible model...

Jan 19 1.3K 20

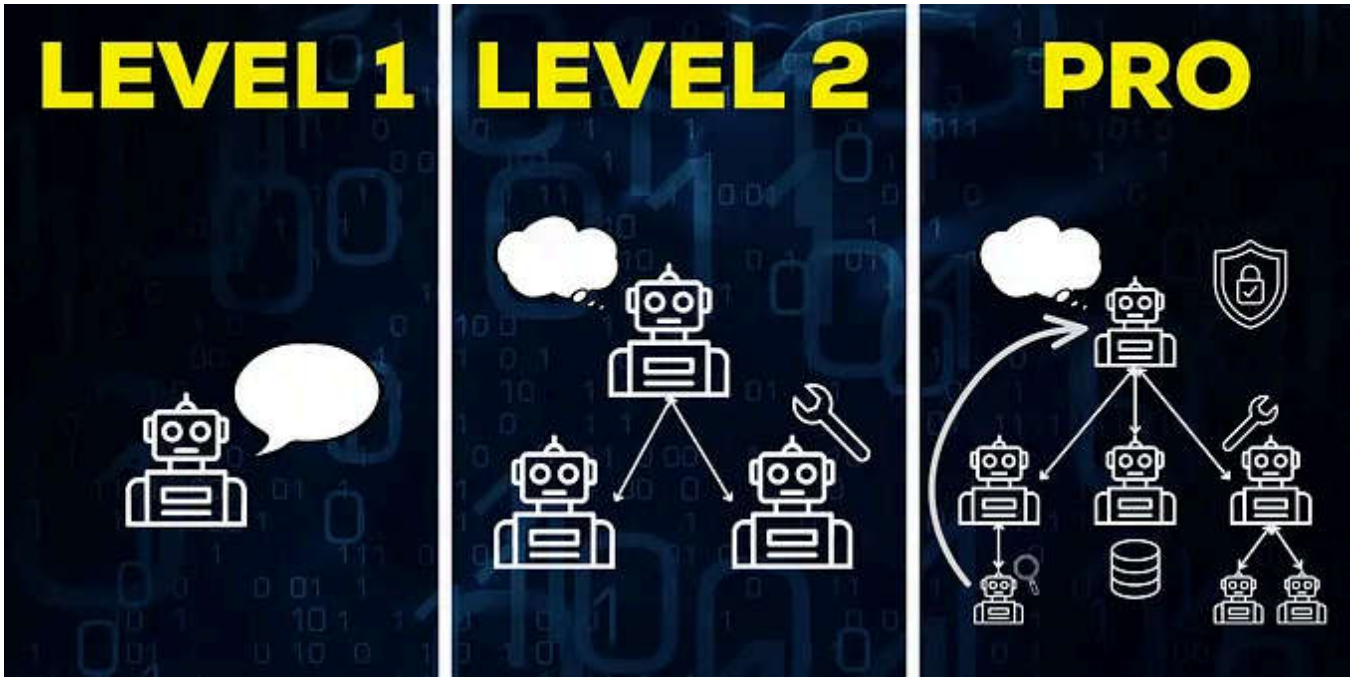


In Python in Plain English by Tarun Singh

Beyond Hybrid RAG That Actually Works: Vector + BM25 + GraphRAG + Reranking in Python (Full Code)

If you're already using GraphRAG + Vector RAG, you'll ahead of most people.

Jan 18 30

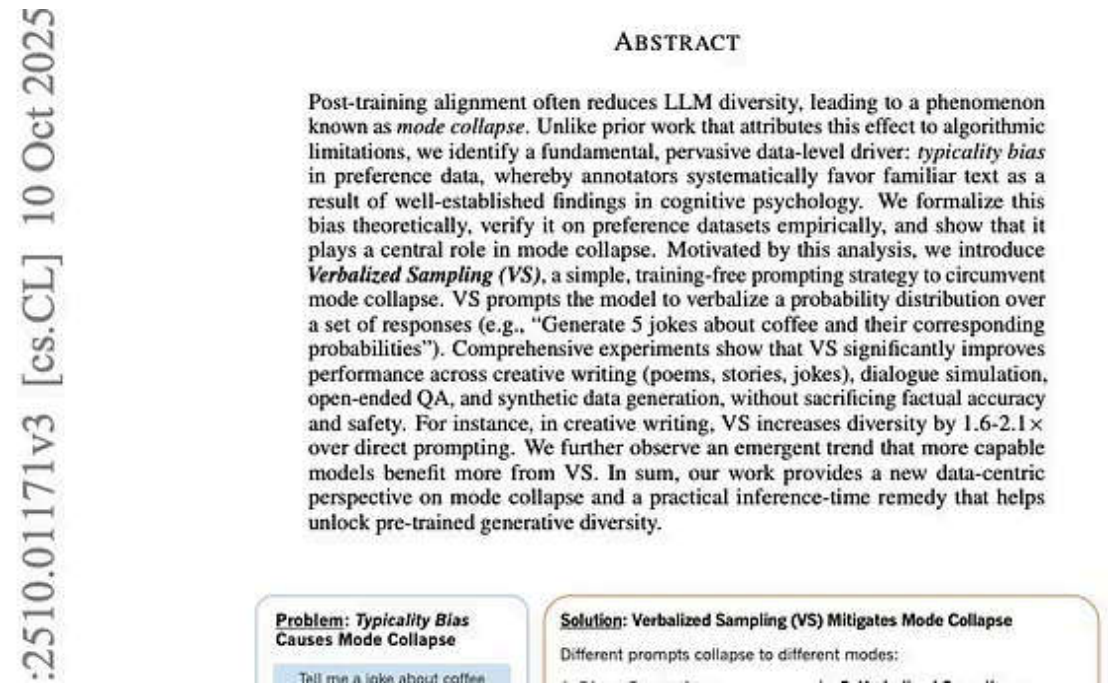


In Data Science Collective by Marina Wyss - Gratitude Driven

AI Agents: Complete Course

From beginner to intermediate to production.

Dec 6, 2025 3K 97



:2510.01171v3 [cs.CL] 10 Oct 2025

ABSTRACT

Post-training alignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce ***Verbalized Sampling (VS)***, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., "Generate 5 jokes about coffee and their corresponding probabilities"). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.

Problem: *Typicality Bias* Causes Mode Collapse

Tell me a joke about coffee

Solution: Verbalized Sampling (VS) Mitigates Mode Collapse

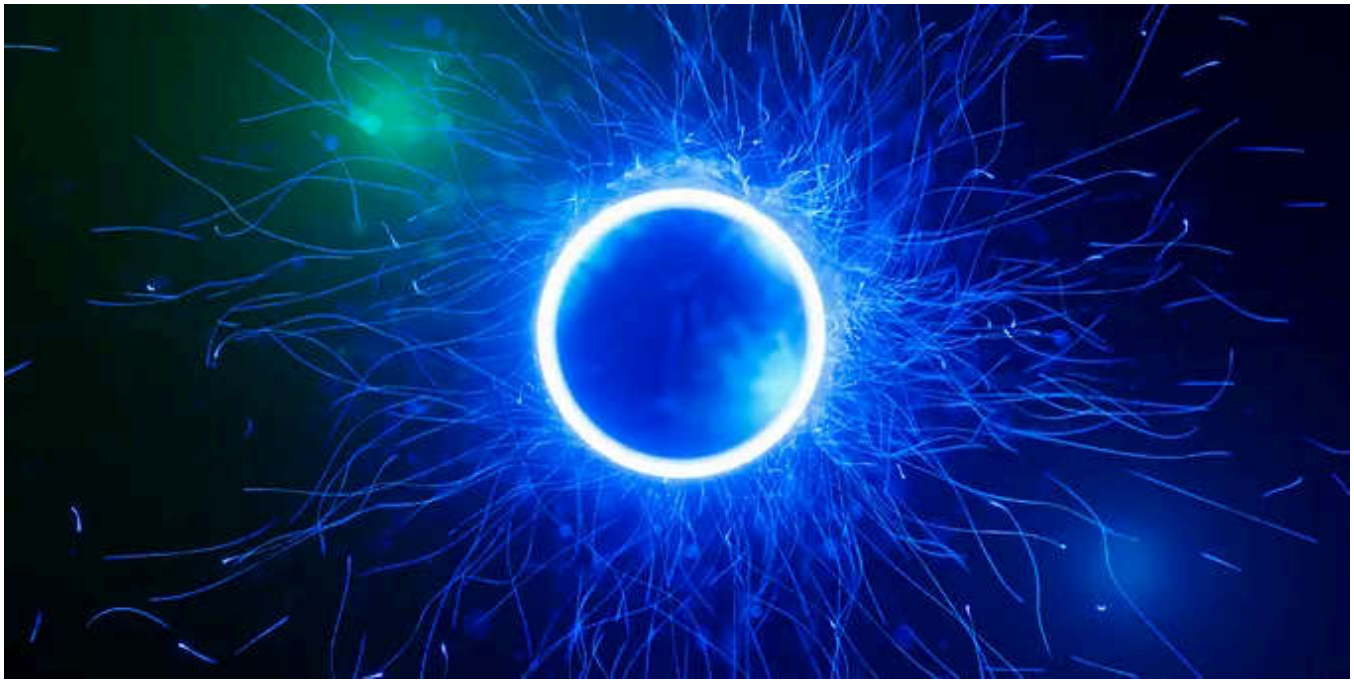
Different prompts collapse to different modes:

In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can't Believe It Worked)

ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model—no training required...

Oct 20, 2025 23K 595



Will Lockett

The AI Bubble Is About To Burst, But The Next Bubble Is Already Growing

Techbros are preparing their latest bandwagon.

Sep 14, 2025 22K 923

See more recommendations